

Jordan Isaac

CS499

CS 499

Milestone Three Enhancement Two: Algorithms and Data Structures

Artifact: CS465 Travlr Getaways

Brief Description of the Artifact

The artifact I selected for the algorithms and data structures enhancement is my CS465 Travlr Getaways full-stack web application. This project was originally developed during the Full Stack Development course and functions as a travel booking and trip management system. The application allows public users to browse travel destinations while authenticated administrators can create, update, and delete trip information through a secured Angular administrative interface. The application was developed using Angular, Node.js, Express, MongoDB, and Mongoose. Communication between the frontend and backend occurs through a RESTful API architecture, while MongoDB stores trip information using document-based collections. The application utilizes JavaScript objects, arrays, JSON responses, and MongoDB documents to manage trip data throughout the system while asynchronous processing supports communication between the frontend, backend, and database layers. I selected this artifact because it provides a strong example of how algorithms and data structures are implemented within a modern full-stack application.

Justification for Inclusion in the ePortfolio

I selected this artifact for inclusion in my ePortfolio because it demonstrates my ability to analyze existing software systems and improve performance through algorithmic enhancements and data structure optimization. During the code review process, I evaluated how trip records were retrieved, processed, and displayed throughout the application. The review identified several opportunities to improve scalability and efficiency. The original implementation retrieved all trip records from the database whenever trip information was requested. While this approach worked for small datasets, it would become increasingly inefficient as the application expanded and additional trip records were added.

To address this limitation, I implemented pagination functionality within the backend API. Pagination retrieves smaller subsets of records by utilizing page numbers, record limits, and skip calculations rather than returning the entire collection. This enhancement significantly reduces unnecessary database retrieval and improves scalability. I also implemented search functionality using MongoDB regular expression queries, allowing users to retrieve specific trip records based on trip names instead of processing the full collection. In addition, I implemented sorting functionality that enables trip records to be organized according to selected fields. Together, these enhancements improve performance, reduce database workload, and create a more efficient user experience while demonstrating practical applications of algorithms and data structures within a full-stack environment.

Course Outcomes

This enhancement supports several Computer Science program outcomes. First, it demonstrates my ability to design, develop, and evaluate computing solutions using algorithmic thinking and modern software development practices. The implementation of pagination, search, and sorting functionality required analyzing system performance and selecting appropriate techniques to improve efficiency. Second, the enhancement demonstrates my ability to apply well-founded computing practices to improve scalability and maintainability. Pagination reduces unnecessary processing by limiting database retrieval, while search and sorting algorithms provide more efficient access to data collections. Finally, the enhancement demonstrates my ability to evaluate existing software systems, identify areas for improvement, and implement optimized solutions that enhance performance and usability. Compared to my original planning, the enhancement evolved beyond simple database optimization and became a larger improvement involving scalable data retrieval techniques, query optimization, and improved collection management.

Reflection on the Enhancement Process

The enhancement process strengthened my understanding of algorithms, data structures, and database performance optimization within full-stack applications. One of the most valuable lessons involved understanding how inefficient data retrieval can impact scalability as applications grow. Implementing pagination demonstrated how relatively simple algorithmic improvements can significantly reduce system workload and improve performance. I also gained experience implementing search functionality using MongoDB regular expressions and learned

how optimized database queries can improve both efficiency and user experience. Another important lesson involved coordinating enhancements across multiple application layers, including the Angular frontend, Express backend, and MongoDB database. Ensuring that pagination, searching, and sorting functionality operated consistently across the entire application required extensive testing and debugging. Overall, this enhancement strengthened my understanding of data retrieval algorithms, database query optimization, asynchronous processing, RESTful API development, and scalable software architecture while demonstrating substantial progress toward several Computer Science program outcomes.

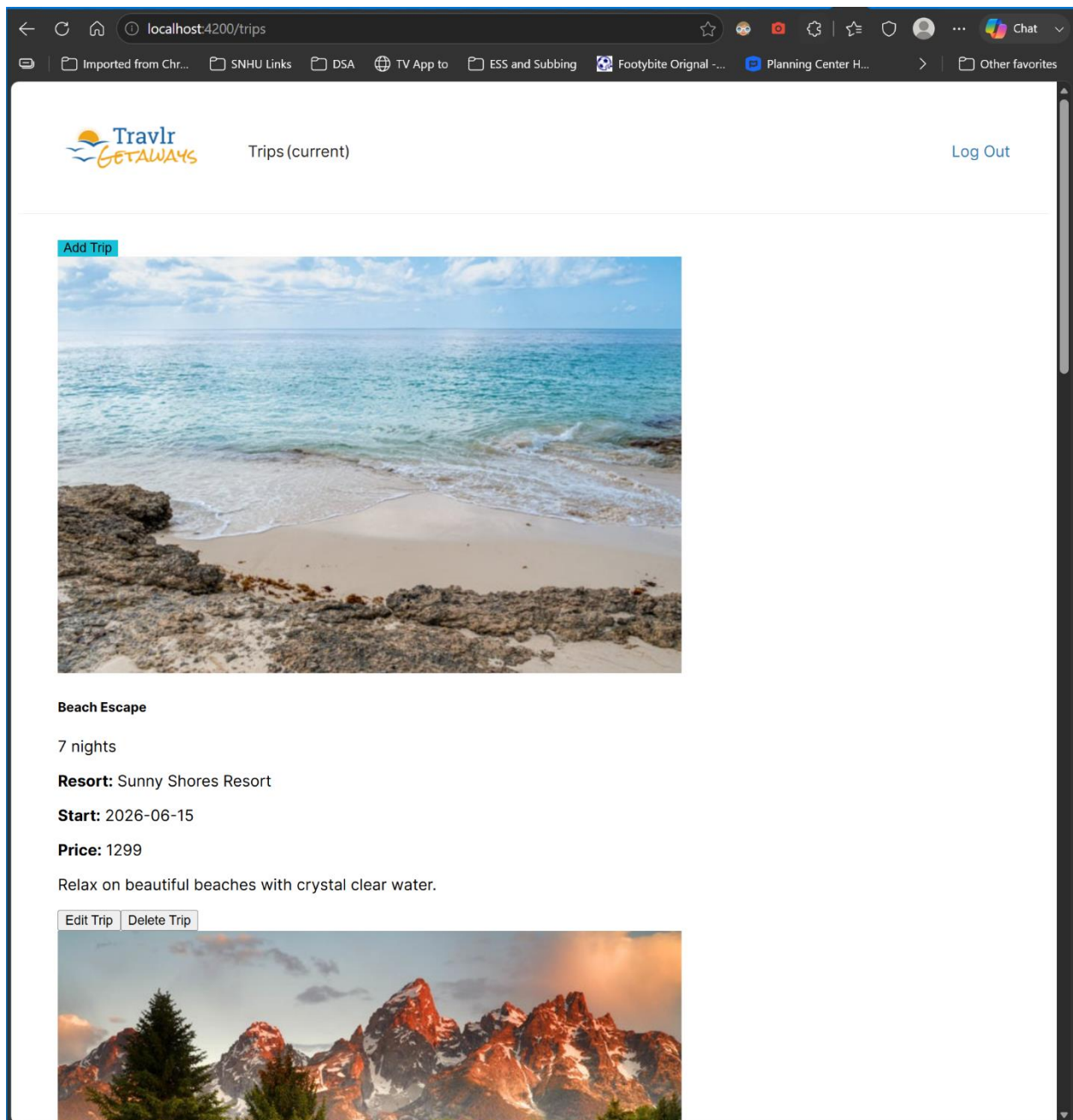


Figure 1. Original Trip Collection Display

This screenshot shows the Trip List component displaying trip records retrieved from the MongoDB database. The Angular frontend dynamically renders trip information received through the REST API. This represents the baseline functionality before search and pagination are applied.

localhost:4200/#/trips

Imported from Chr...SNHU LinksDSA TV App toESS and SubbingFootybite Original ...Planning Center H...Other favorites

Chat

Trips (current)

Log Out

Add Trip



Beach Escape

7 nights

Resort: Sunny Shores Resort

Start: 2026-06-15

Price: 1299

Relax on beautiful beaches with crystal clear water.

Edit Trip

Delete Trip



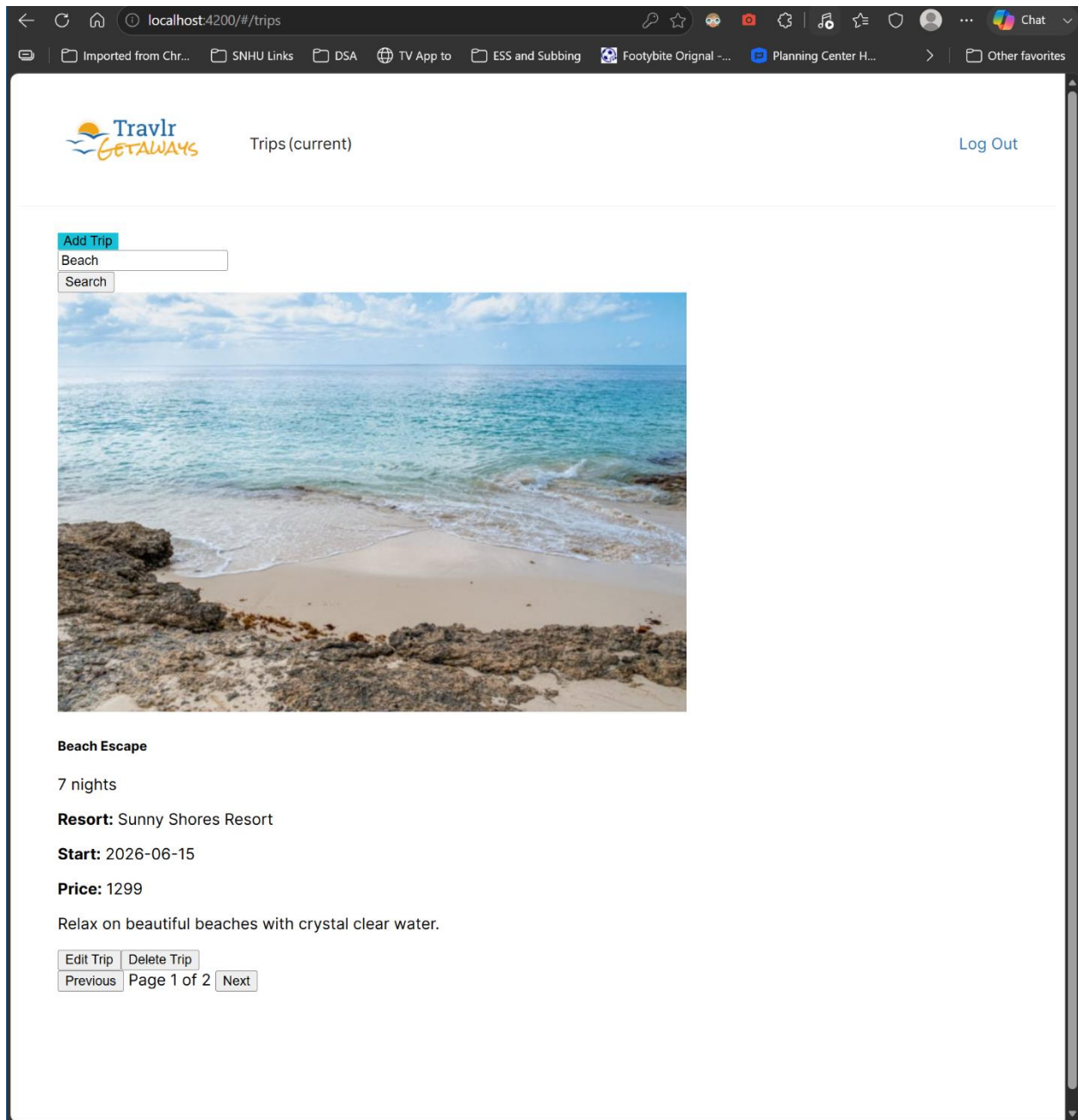


Figure 2 - 3. Search Algorithm Enhancement Filtering Trip Records

This screenshot demonstrates the search enhancement implemented during the algorithms and data structures modification process. The search functionality utilizes MongoDB regular expression matching to retrieve only records that contain the specified search term. This

enhancement improves efficiency by reducing unnecessary data processing and allows users to quickly locate specific trip records.

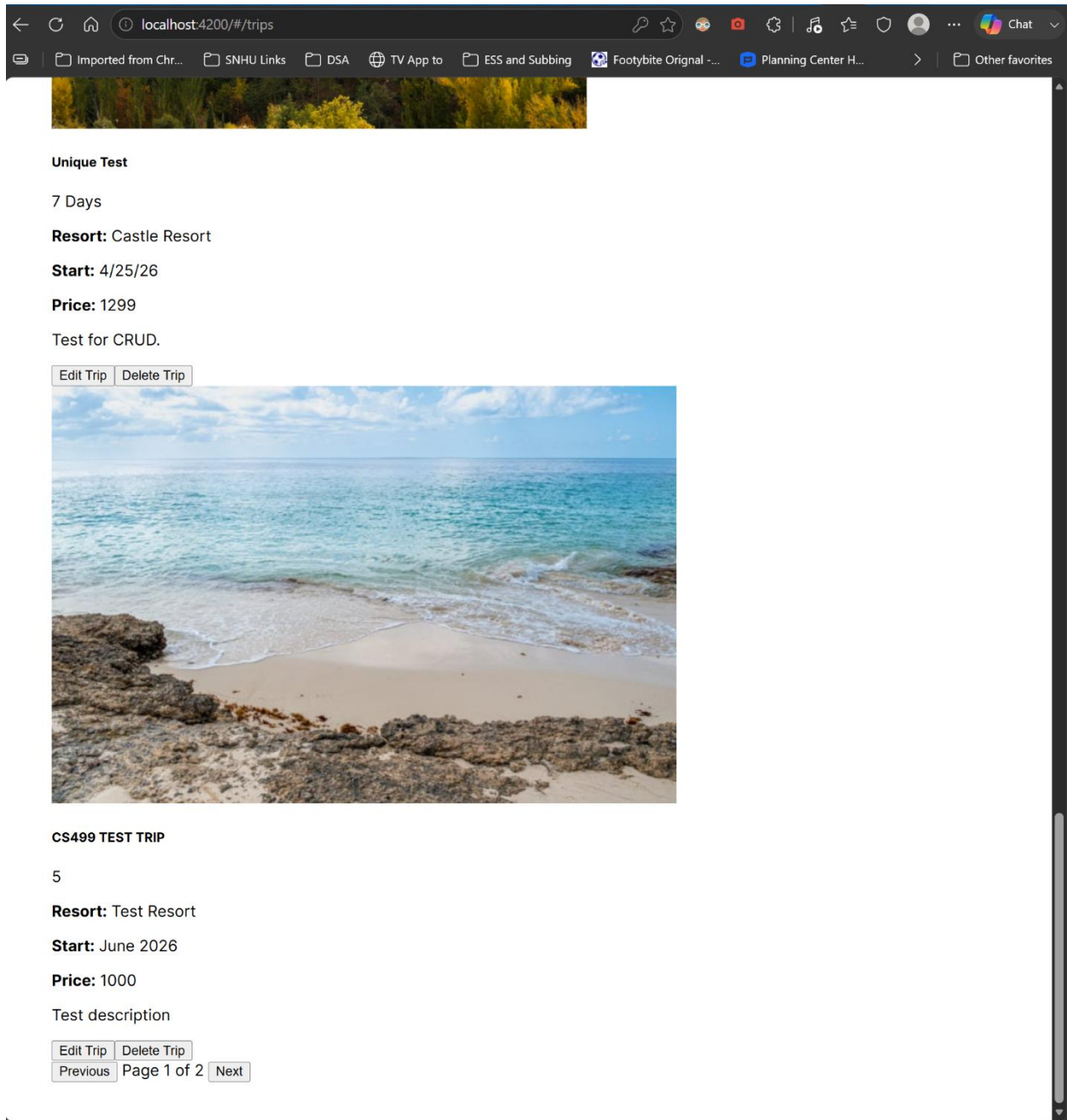


Figure 4. Pagination Algorithm – First Page Results

This screenshot demonstrates the pagination enhancement displaying the first page of trip results. Instead of retrieving the entire database collection, the application retrieves only a subset of records based on the selected page number and page size. This improves scalability and reduces database workload.

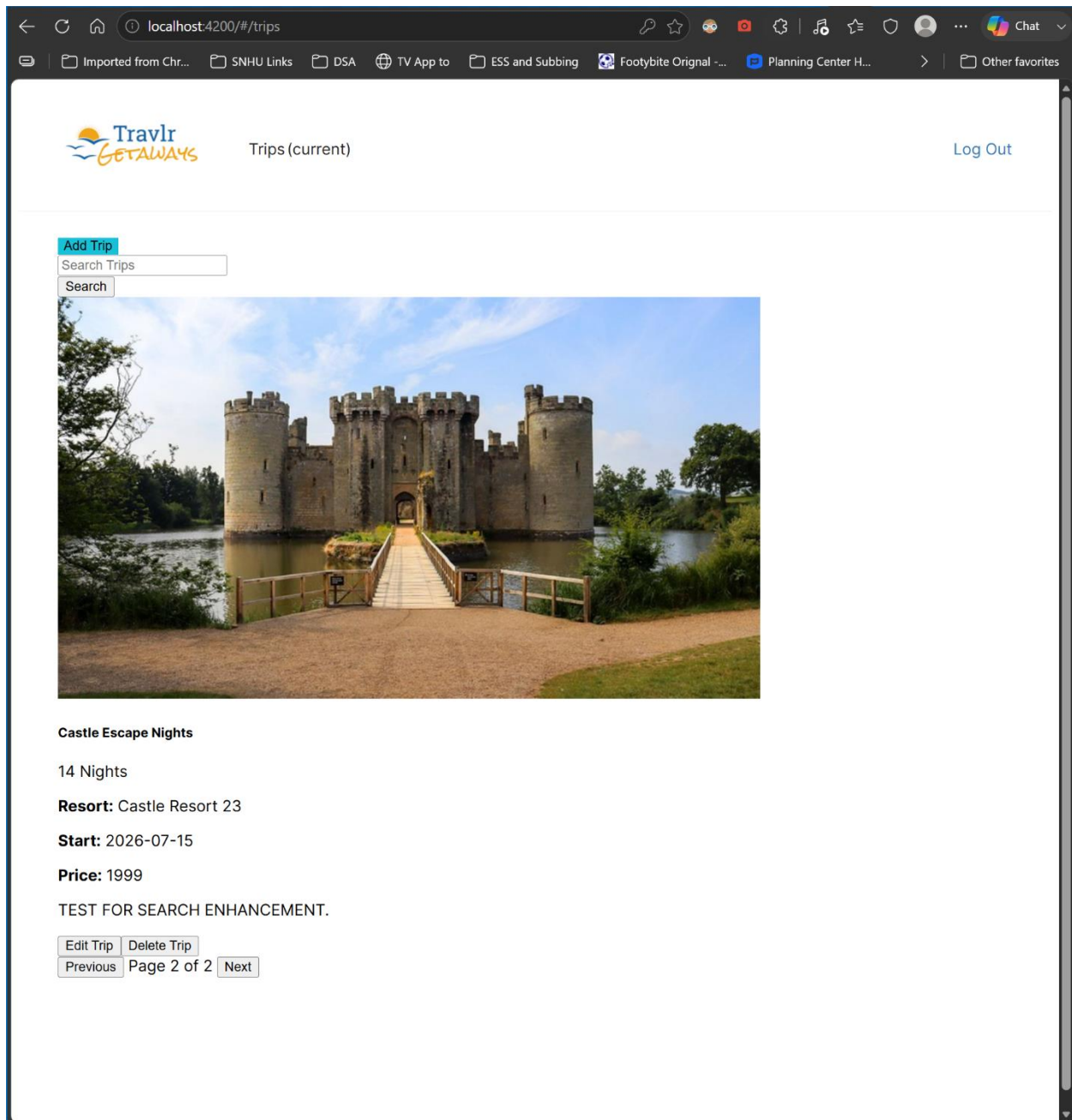


Figure 5. Pagination Algorithm – Second Page Results

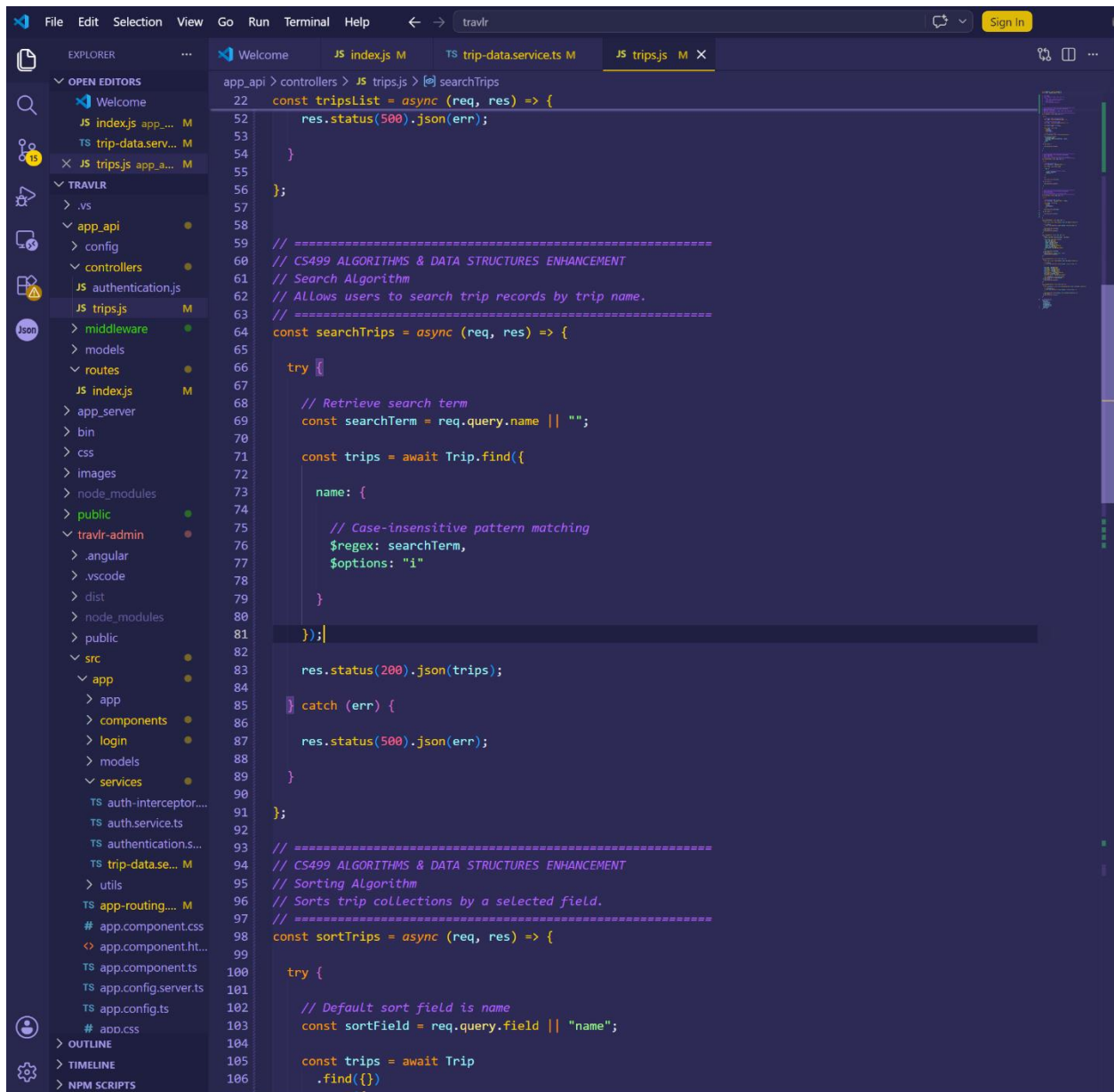
This screenshot demonstrates the second page of paginated results. The application uses skip and limit calculations to retrieve a different subset of records from the database. This enhancement demonstrates practical implementation of collection management and efficient data retrieval algorithms.

The screenshot shows a Visual Studio Code editor with a project named 'travlr'. The Explorer sidebar on the left shows the file structure, with 'app_api > controllers > JS tripsjs' selected. The main editor displays the 'tripsjs.js' file, which implements a pagination algorithm. The code starts with a comment 'OLD CODE:' and shows a simple async function that returns all trip records. Below this, a new section titled 'CS499 ALGORITHMS & DATA STRUCTURES ENHANCEMENT' shows a more complex implementation. This enhanced version calculates the page number from the query string, sets a default page of 1 and a limit of 5 records per page. It then calculates the number of records to skip based on the page and limit. The code uses the MongoDB 'find' method with 'skip' and 'limit' options to retrieve the records. It also calculates the total number of records using 'countDocuments' and returns the current page, total pages, total records, and the trip records in the response. Error handling is included for both the database query and the response status.

```
4
5 // OLD CODE:
6 // const tripsList = async (req, res) => {
7 //   try {
8 //     const trips = await Trip.find({});
9 //     res.status(200).json(trips);
10 //   } catch (err) {
11 //     res.status(500).json(err);
12 //   }
13 // };
14
15
16 // =====
17 // CS499 ALGORITHMS & DATA STRUCTURES ENHANCEMENT
18 // Pagination Algorithm
19 // Retrieves trip records in pages rather than returning
20 // the entire collection.
21 // =====
22 const tripsList = async (req, res) => {
23
24   try {
25
26     // Get page number from query string
27     const page = parseInt(req.query.page) || 1;
28
29     // Number of records per page
30     const limit = parseInt(req.query.limit) || 5;
31
32     // Calculate records to skip
33     const skip = (page - 1) * limit;
34
35     const trips = await Trip
36       .find({})
37       .skip(skip)
38       .limit(limit);
39
40     // Count total records
41     const totalTrips = await Trip.countDocuments();
42
43     res.status(200).json({
44       currentPage: page,
45       totalPages: Math.ceil(totalTrips / limit),
46       totalTrips,
47       trips
48     });
49   } catch (err) {
50
51     res.status(500).json(err);
52   }
53
54 };
55
56 // =====
57
58
59
```

Figure 6. Backend Pagination Algorithm Implementation

This screenshot displays the backend pagination implementation within the Express controller. The algorithm calculates page size, record limits, and skipped records before querying MongoDB. This enhancement reduces unnecessary retrieval of large collections and improves overall application performance.



```
22 const tripsList = async (req, res) => {
23   res.status(500).json(err);
24 }
25
26 };
27
28 // =====
29 // CS499 ALGORITHMS & DATA STRUCTURES ENHANCEMENT
30 // Search Algorithm
31 // Allows users to search trip records by trip name.
32 // =====
33 const searchTrips = async (req, res) => {
34   try {
35     // Retrieve search term
36     const searchTerm = req.query.name || "";
37
38     const trips = await Trip.find({
39       name: {
40         // Case-insensitive pattern matching
41         $regex: searchTerm,
42         $options: "i"
43       }
44     });
45
46     res.status(200).json(trips);
47   } catch (err) {
48     res.status(500).json(err);
49   }
50 };
51
52 // =====
53 // CS499 ALGORITHMS & DATA STRUCTURES ENHANCEMENT
54 // Sorting Algorithm
55 // Sorts trip collections by a selected field.
56 // =====
57 const sortTrips = async (req, res) => {
58   try {
59     // Default sort field is name
60     const sortField = req.query.field || "name";
61
62     const trips = await Trip
63       .find({})
64       .sort({ [sortField]: 1 })
65       .limit(10);
66
67     res.status(200).json(trips);
68   } catch (err) {
69     res.status(500).json(err);
70   }
71 };
72
```

Figure 7. MongoDB Search Algorithm Implementation

This screenshot displays the searchTrips controller function. The implementation uses MongoDB regular expression matching with case-insensitive searching to locate records based on user-provided search criteria. This enhancement demonstrates practical application of search algorithms within a database environment.

```
9 @Injectable({
10   providedIn: 'root'
11 })
12 export class TripDataService {
13   baseUrl = 'http://localhost:3000/api';
14
15   constructor(
16     private http: HttpClient,
17     @Inject(BROWSER_STORAGE) private storage: Storage
18   ) {}
19
20   getTrips(): Observable<Trip[]> {
21     return this.http.get<Trip[]>(`${this.baseUrl}/trips`);
22   }
23
24   // =====
25   // CS499 ALGORITHMS & DATA STRUCTURES ENHANCEMENT
26   // Pagination Algorithm
27   // =====
28   getTripsPaginated(page: number, limit: number): Observable<any> {
29
30     return this.http.get<any>(`
31       ${this.baseUrl}/trips?page=${page}&limit=${limit}`
32     );
33   }
34
35   // =====
36   // CS499 ALGORITHMS & DATA STRUCTURES ENHANCEMENT
37   // Search Algorithm
38   // =====
39   searchTrips(searchTerm: string): Observable<Trip[]> {
40
41     return this.http.get<Trip[]>(`
42       ${this.baseUrl}/trips/search?name=${searchTerm}`
43     );
44   }
45
46   // =====
47   // CS499 ALGORITHMS & DATA STRUCTURES ENHANCEMENT
48   // Sorting Algorithm
49   // =====
50   sortTrips(field: string): Observable<Trip[]> {
51
52     return this.http.get<Trip[]>(`
53       ${this.baseUrl}/trips/sort?field=${field}`
54     );
55   }
56
57   getTrip(code: string): Observable<Trip> {
58     return this.http.get<Trip>(`${this.baseUrl}/trips/${code}`);
59   }
60
61   addTrip(trip: Trip): Observable<Trip> {
```

Figure 8. Angular Service Enhancements Supporting Data Retrieval

This screenshot displays the Angular service methods developed to support pagination, searching, and sorting operations. These reusable service functions communicate with the backend API and provide a structured approach for retrieving data collections throughout the application.

File Edit Selection View Go Run Terminal Help

travlr

Sign In

EXPLORER

OPEN EDITORS

travlr-admin > src > app > components > trip-list > TS trip-list.component.ts > TripListComponent > constructor

JS index.js app... M

TS trip-data.serv... M

TS trip-list.comp... M

JS trips.js app... M

TRAVLR

app_api

controllers

JS authentication.js

trips.js

middleware

models

routes

JS index.js

app_server

bin

css

images

node_modules

public

travlr-admin

angular

vscode

dist

node_modules

public

src

app

app

components

login

navbar

trip-card

trip-form

trip-list

trip-list.compo...

trip-list.co... M

trip-list.co... M

login

models

services

TS auth-interceptor...

TS auth.service.ts

TS authentication.s...

TS trip-data.se... M

utils

OUTLINE

TIMELINE

NPM SCRIPTS

12

13

14

15

16

17

18

19

20

21

22

23

24

25

26

27

28

29

30

31

32

33

34

35

36

37

38

39

40

41

42

43

44

45

46

47

48

49

50

51

52

53

54

55

56

57

58

59

60

61

62

63

64

65

66

67

68

69

70

71

72

73

74

75

76

77

78

79

80

81

82

83

84

85

86

87

88

89

90

91

92

93

94

95

96

97

98

99

100

101

102

103

104

105

106

107

108

109

110

111

112

113

114

115

116

117

118

119

120

121

122

123

124

125

126

127

128

129

130

131

132

133

134

135

136

137

138

139

140

141

142

143

144

145

146

147

148

149

150

151

152

153

154

155

156

157

158

159

160

161

162

163

164

165

166

167

168

169

170

171

172

173

174

175

176

177

178

179

180

181

182

183

184

185

186

187

188

189

190

191

192

193

194

195

196

197

198

199

200

201

202

203

204

205

206

207

208

209

210

211

212

213

214

215

216

217

218

219

220

221

222

223

224

225

226

227

228

229

230

231

232

233

234

235

236

237

238

239

240

241

242

243

244

245

246

247

248

249

250

251

252

253

254

255

256

257

258

259

260

261

262

263

264

265

266

267

268

269

270

271

272

273

274

275

276

277

278

279

280

281

282

283

284

285

286

287

288

289

290

291

292

293

294

295

296

297

298

299

300

301

302

303

304

305

306

307

308

309

310

311

312

313

314

315

316

317

318

319

320

321

322

323

324

325

326

327

328

329

330

331

332

333

334

335

336

337

338

339

340

341

342

343

344

345

346

347

348

349

350

351

352

353

354

355

356

357

358

359

360

361

362

363

364

365

366

367

368

369

370

371

372

373

374

375

376

377

378

379

380

381

382

383

384

385

386

387

388

389

390

391

392

393

394

395

396

397

398

399

400

401

402

403

404

405

406

407

408

409

410

411

412

413

414

415

416

417

418

419

420

421

422

423

424

425

426

427

428

429

430

431

432

433

434

435

436

437

438

439

440

441

442

443

444

445

446

447

448

449

450

451

452

453

454

455

456

457

458

459

460

461

462

463

464

465

466

467

468

469

470

471

472

473

474

475

476

477

478

479

480

481

482

483

484

485

486

487

488

489

490

491

492

493

494

495

496

497

498

499

500

501

502

503

504

505

506

507

508

509

510

511

512

513

514

515

516

517

518

519

520

521

522

523

524

525

526

527

528

529

530

531

532

533

534

535

536

537

538

539

540

541

542

543

544

545

546

547

548

549

550

551

552

553

554

555

556

557

558

559

560

561

562

563

564

565

566

567

568

569

570

571

572

573

574

575

576

577

578

579

580

581

582

583

584

585

586

587

588

589

590

591

592

593

594

595

596

597

598

599

600

601

602

603

604

605

606

607

608

609

610

611

612

613

614

615

616

617

618

619

620

621

622

623

624

625

626

627

628

629

630

631

632

633

634

635

636

637

638

639

640

641

642

643

644

645

646

647

648

649

650

651

652

653

654

655

656

657

658

659

660

661

662

663

664

665

666

667

668

669

670

671

672

673

674

675

676

677

678

679

680

681

682

683

684

685

686

687

688

689

690

691

692

693

694

695

696

697

698

699

700

701

702

703

704

705

706

707

708

709

710

711

712

713

714

715

716

717

718

719

720

721

722

723

724

725

726

727

728

729

730

731

732

733

734

735

736

737

738

739

740

741

742

743

744

745

746

747

748

749

750

751

752

753

754

755

756

757

758

759

760

761

762

763

764

765

766

767

768

769

770

771

772

773

774

775

776

777

778

779

780

781

782

783

784

785

786

787

788

789

790

791

792

793

794

795

796

797

798

799

800

801

802

803

804

805

806

807

808

809

810

811

812

813

814

815

816

817

818

819

820

821

822

823

824

825

826

827

828

829

830

831

832

833

834

835

836

837

838

839

840

841

842

843

844

845

846

847

848

849

850

851

852

853

854

855

856

857

858

859

860

861

862

863

864

865

866

867

868

869

870

871

872

873

874

875

876

877

878

879

880

881

882

883

884

885

886

887

888

889

890

891

892

893

894

895

896

897

898

899

900

901

902

903

904

905

906

907

908

909

910

911

912

913

914

915

916

917

918

919

920

921

922

923

924

925

926

927

928

929

930

931

932

933

934

935

936

937

938

939

940

941

942

943

944

945

946

947

948

949

950

951

952

953

954

955

956

957

958

959

960

961

962

963

964

965

966

967

968

969

970

971

972

973

974

975

976

977

978

979

980

981

982

983

984

985

986

987

988

989

990

991

992

993

994

995

996

997

998

999

1000

module7+

1

0

Jordan I (1 month ago)

Ln 30, Col 46

Spaces: 2

UTF-8

```
12 export class TripListComponent implements OnInit {
13
14 }
15
16 // =====
17 // CS499 ALGORITHMS & DATA STRUCTURES ENHANCEMENT
18 // Search Algorithm
19 // Searches trip collections by trip name.
20 // =====
21 searchTrips(): void {
22
23     if (!this.searchTerm.trim()) {
24
25         this.loadTrips();
26
27         return;
28     }
29
30     this.tripDataService
31       .searchTrips(this.searchTerm)
32       .subscribe({
33
34         next: (data: Trip[]) => {
35
36             this.trips = data;
37
38         },
39
40         error: (err: any) => {
41
42             console.error(
43               'Search error:',
44               err
45             );
46
47         }
48       });
49 }
50
51 // =====
52 // CS499 ALGORITHMS & DATA STRUCTURES ENHANCEMENT
53 // Pagination Navigation
54 // =====
55 nextPage(): void {
56
57     if (this.currentPage < this.totalPages) {
58
59         this.currentPage++;
60
61         this.loadTrips();
62
63     }
64 }
65
66 }
```

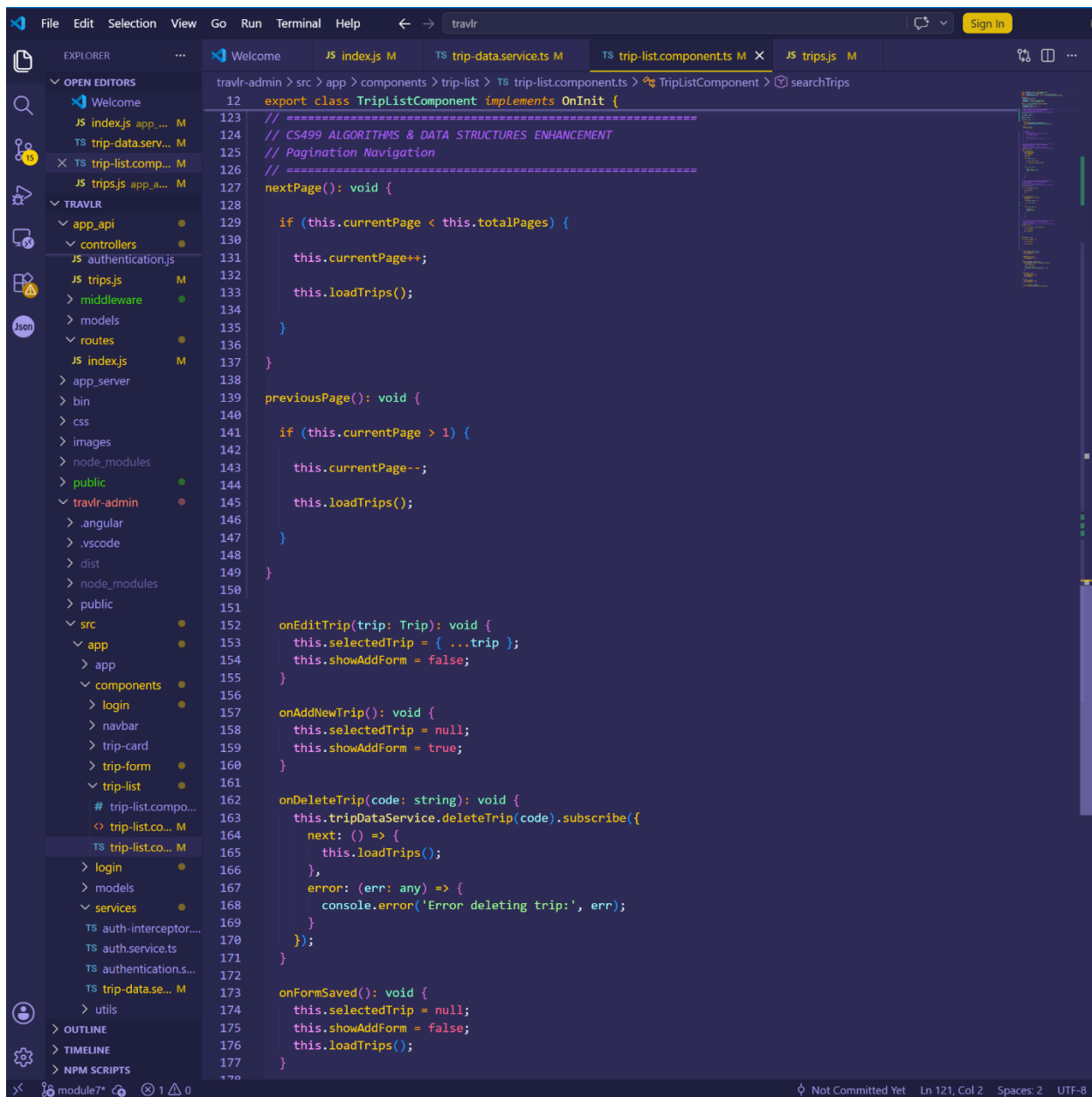



Figure 9-11. Frontend Search and Pagination Logic

This screenshot displays the frontend implementation responsible for processing pagination and search requests. The component manages user interactions, retrieves filtered data through Angular services, and dynamically updates the displayed trip collection. This enhancement improves both usability and application performance.

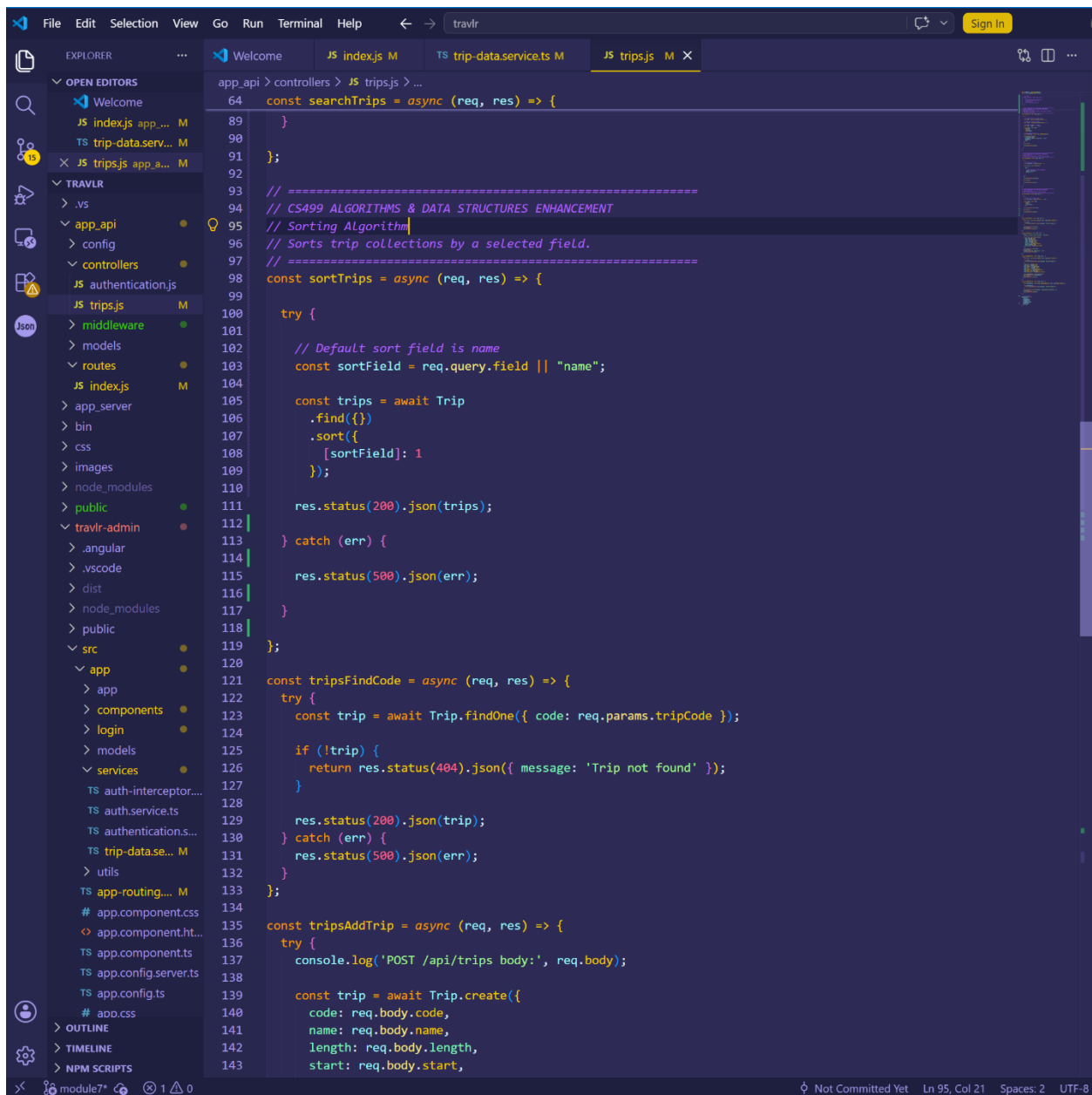


Figure 12 . Trip Sorting Algorithm Enhancement

This screenshot demonstrates the sorting algorithm enhancement implemented within the Travlr Getaways application. The enhancement allows trip records to be dynamically sorted based on a selected field provided through the query string. A default sort field of "name" is used when no specific field is supplied by the user.

The algorithm retrieves trip records from MongoDB using the Mongoose find() method and applies the sort() function to organize the results in ascending order. By utilizing a dynamic property ([sortField]: 1), the application can sort records by different attributes without requiring multiple hard-coded sorting methods. This approach improves flexibility, maintainability, and scalability within the application.

Prior to this enhancement, trip records were returned in their natural database order, which could vary depending on insertion sequence and database operations. The addition of the sorting algorithm provides users with a more organized view of trip information and improves overall usability. This enhancement demonstrates the practical application of algorithms and data structures concepts by improving data retrieval efficiency and supporting user-driven organization of information.